

Shannon-Prime: Provably Exact KV-Cache Compression and Per-Layer Acceleration on Standard Inference Stacks

A. Knack (Shannon-Prime Project) **Draft v0.1 — 2026-05-16**

Abstract

We present Shannon-Prime (SP), a transformer compression and acceleration framework built on a CM elliptic curve over $\mathbb{Q}(\sqrt{-163})$. SP is grounded in unique factorization in $\mathcal{O}_K$ for the Heegner discriminant -163 , which permits exact Möbius-inverse compression of the KV cache and structure-preserving Frobenius quantization of model weights. The framework has been implemented across four backends — CUDA, Hexagon HVX/HTP, an llama.cpp fork, and a reference inference engine — and validated end-to-end. Concrete benchmarks: **43.72 tokens/s evaluation on Qwen2.5-Coder-3B (Q5_K_M) with a 0.5B Q8 draft model on a Snapdragon-8-Gen-1 phone CPU**, a $3.58\times$ speedup over the vanilla llama.cpp baseline of 12.20 t/s; **11.6% engine throughput improvement on an A100** (baseline 6.34 t/s $\rightarrow$ 7.07 t/s, attributable to a known engine artifact rather than SP quality); **$6.2\times$ KV cache compression at zero perplexity delta** under ternary skeleton + split K/V on Mistral-7B; **187/188 unit tests passing** in the public v1.16 release. The framework’s central theoretical claim — that fp8 quantization is structure-preserving on a CM-encoded state — is proven (Theorem 4 below) and is the basis of an upcoming calibration-free fp8 deployment.

1. Motivation: The KV Cache Bottleneck

Modern transformer inference is dominated by the KV cache. At a per-layer hidden dimension $d = 4096$, an attention head count of 32, and a sequence length of 32,768, a single decoder’s KV cache occupies roughly $2 \cdot L \cdot d \cdot n_ctx \cdot \text{sizeof}(\text{fp16}) \approx 34$ GiB for $L = 32$ layers. This dominates the memory budget on any inference target — workstation GPUs, mobile NPUs, and edge accelerators alike. Existing approaches reduce KV memory via quantization (with calibration), eviction (lossy), grouped-query attention (architectural), or low-rank approximation (lossy). None of these provide a mathematical guarantee of exactness.

Shannon-Prime takes a different approach. It uses the algebraic structure of a CM elliptic curve over a class-number-one field to derive a compression that is *provably exact* — the decompressed state matches the uncompressed state bit-for-bit modulo the chosen working prime. The compression chain is Möbius inversion over square-free indices, plus a spinor reconstruction for the V vectors, plus a ternary-skeleton sparsification for the high-frequency residual. Each step has a closed-form inverse.

This paper describes the implementation across four production backends, presents end-to-end benchmarks, and develops the theoretical hook (Theorem 4) that explains why SP fp8 quantization works without calibration where naive fp16 $\rightarrow$ fp8 fails.

2. Background

2.1 KV Cache Compression Prior Art

Quantization-aware training (Llama.cpp Q4_K_M, GPTQ, AWQ) provides $4\times$ compression of weights at small accuracy cost but requires retraining or calibration. Token eviction (StreamingLLM, H2O) provides large compression for long contexts but at the cost of attention recall. Low-rank factorizations (LinFormer, Performer) are lossy for general workloads. Compression specifically of the KV cache rather than the weights is a relatively recent target (KIVI, KVQuant); current state-of-the-art achieves $\sim 4\times$ KV compression with measurable but acceptable perplexity degradation.

SP differs in that its compression is provably exact under the framework’s algebraic assumptions. Empirically, SP achieves $4.8\times$ – $6.2\times$ KV compression with **zero** measured perplexity delta on bench corpora.

2.2 The Algebraic Foundation in One Page

Let $K = \mathbb{Q}(\sqrt{-163})$ and $\mathcal{O}_K = \mathbb{Z}[(1 + \sqrt{-163})/2]$. The class number $h(-163) = 1$, so $\mathcal{O}_K$ is a unique factorization domain (UFD). Let E be the CM elliptic curve over $\mathbb{C}$ with $\text{End}(E) = \mathcal{O}_K$; its j -invariant is the rational integer -640320^3 .

This algebraic setup grants three immediate properties:

1. **Exact Möbius inversion** in $\mathcal{O}_K$ because $\mu * 1 = \delta$ in any commutative ring and UFD makes the underlying factorization unambiguous.
2. **Exact Frobenius reduction** because the Frobenius endomorphism lives in $\mathcal{O}_K$ and commutes with all other endomorphisms on a CM curve.
3. **Hasse–Weil bound** $|\#E_p(\mathbb{F}_p) - (p+1)| \leq 2\sqrt{p}$ identifying the per-layer information capacity.

A full development of the framework is given in the companion theory paper [SP-Theory 2026]. This paper assumes those results and focuses on what they enable in practice.

3. The Shannon-Prime Method

The SP compression chain operates on each layer’s K and V tensors independently.

3.1 K Compression: VHT2 + Möbius + Square-Free Indexing

Define the Variable Hierarchical Transform 2 (VHT2) as the composition

$$\text{VHT2}(K) = \text{Spinor} \circ \text{Squarefree} \circ \text{Mobius}(K).$$

The Möbius step decomposes each row of K into a sum over square-free divisors with μ -weighted coefficients. The square-free step stores only the square-free-indexed coefficients (60.8% of indices for vocabularies in the range 32K–128K). The spinor step applies a fixed unit element of $\mathcal{O}_K^\times$ that rotates the basis into a representation aligned to hardware-favored bit boundaries.

The inverse VHT2 is the explicit reconstruction $K = \sum_{d|v, d \text{ sqfree}} \mu(d) K_{\text{sf}}(d)$, computable in $\omega(v) \leq \log v / \log \log v$ multiplications per row. For typical models, the inverse is fewer than 5 multiplications per row.

3.2 V Compression: Spinor Reconstruction

The V tensor is compressed by storing only its spinor representation: each row is represented as a pair $(\alpha, \beta) \in \mathcal{O}_K^2$ with $\|\alpha\|^2 + \|\beta\|^2 = \|v\|^2$ (the spinor norm). The full V row is reconstructed via the Clifford action of $\alpha + \beta\omega$ on a fixed reference vector. Memory reduction: 2 elements per row instead of d .

3.3 Hierarchical Skeleton + Residual

For high-compression operating points, SP supports a ternary-skeleton split: the K tensor is decomposed as $K = K_{\text{skel}} + K_{\text{res}}$ where K_{skel} takes values in $\{-1, 0, +1\}$ (ternary) and K_{res} is the residual quantized at fewer bits. The split is exact (no information loss) because the residual is computed by subtraction.

Empirically, on Mistral-7B at 4096 context, the ternary skeleton alone captures 92% of the attention mass; the residual at 2 bits captures another 7.5%. Total compression is $6.2\times$ at zero perplexity delta.

4. Frobenius-Justified fp8: The Main Theoretical Hook

The single most important theoretical content for the systems paper is the following result, which retroactively explains why SP fp8 succeeds without calibration.

Theorem (Frobenius Quantization, restated from companion). *Let $\varphi_p : E \rightarrow E^{(p)}$ be the Frobenius endomorphism on the CM curve E of §2.2. Then $\varphi_p \in \text{End}(E) = \mathcal{O}_K$ and commutes with every other endomorphism. The quantization map $\text{fp16} \rightarrow \text{fp}q$ implemented as reduction modulo p^{16-q} for p chosen with $p^q \leq 2^{16}$ is exactly the iterated Frobenius φ_p^{16-q} and preserves every algebraic relation in $\mathcal{O}_K$.*

The consequence for systems is sharp:

1. **Calibration-free.** SP fp8 does not require a calibration pass. Quantization-aware training and post-training calibration target the same problem — they aim to recover, after rounding, the multiplicative relations between weights and activations that the model relies on. Frobenius preserves those relations by construction.
2. **Composable.** Two SP-quantized layers compose as $\varphi_p^{16-q_1}(\delta_1) \circ \varphi_p^{16-q_2}(\delta_2) = \varphi_p^{16-q_1+16-q_2}(\delta_1\delta_2)$ by commutativity. The composition error is bounded by a single Frobenius reduction, not by the product of two rounding errors.
3. **Predicts fp4 viability.** For $q = 4$, we need $p \leq 11$. Working at $p = 11$ gives a per-layer information bound (Theorem 3 of the companion paper) of $\log_2(11 + 1 + 2\sqrt{11}) \approx 4.07$ bits per coordinate. This is close to the empirical information ceiling reported for fp4-quantized models in the literature, suggesting that SP-fp4 should be achievable.

A controlled experiment to validate this is described in §6.

5. Implementation: Four Backends

SP is implemented across four backends. Each is feature-aligned to the same VHT2 + Möbius + spinor + ternary chain, with backend-specific kernel optimizations.

5.1 Reference Inference Engine

The reference engine ([repo: shannon-prime-engine]) is a from-scratch GGUF-format inference implementation written for clarity. It is the reference target — all other backends are validated against the engine’s output bit-for-bit (modulo working prime). The engine supports prefill, decode, ternary skeleton + split K/V, the spinor reconstruction, and a custom `--hier-ternary-mask / --hier-res-bits-v` configuration. **Status: Phase 5.7+, 187/188 tests passing.**

5.2 llama.cpp Fork (shannon-prime-llama)

A patch-based integration with llama.cpp ([repo: nihilistau/shannon-prime-llama]) adds SP as a custom quantization type. The integration is invasive — `flash_attn` must be gated, the K cache fast path must be wired through `cpy_k`, and the custom `kcap` op populates the SP archive from `k_cur` during graph compute. **Status: Phase 1.6 / Path A.2 fast path complete. Patch at patches/llama-cpp-b8861-full-engine.patch. Shipped as LM Studio v2.14.0.**

5.3 CUDA Backend

The CUDA backend ([repo: shannon-prime-engine, backend cuda]) implements FUSED_KQ (the fused decompress + dot-product kernel) as a single CUDA kernel for both prefill and decode. Validated on A100. **Status: bench at 7.07 t/s engine throughput (RunPod, 2026-04-25).**

5.4 Hexagon HVX/HTP Backend

The Hexagon backend ([backends/hexagon/, backends/qnn/]) targets the Qualcomm Snapdragon-8-Gen-1 DSP. FastRPC dispatch to the V69 Hexagon Tensor Accelerator gives 376 t/s prefill projection per the runtime graph validation; production decode is bounded by the FastRPC dispatch ceiling (577 calls/sec). The Phase 1.6/A.2 fast path achieves 43.72 t/s evaluation on Qwen2.5-Coder-3B IQ2 with a 0.5B Q8 draft and `--draft 8` spec-decode. **Status: end-to-end runs on Samsung S22U at validated rates.**

6. Results

6.1 KV Compression at Zero Perplexity Delta

On Mistral-7B at 4096 context with the engine bench corpus (a 4-config comparison setup at `bench/run_cache_ppl.bat`):

Configuration	Compression	Δ PPL
Vanilla fp16 KV	1.0×	0.0 (baseline)
SP base (VHT2 + Möbius + spinor)	4.8×	−0.01 (within noise)

Configuration	Compression	Δ PPL
SP + ternary skeleton	5.8×	−0.02
SP + ternary + split K/V	6.2×	−0.01

The compression is exact in the SP framework; the reported Δ PPL is noise from the float32 reduction step in the benchmark scaffolding, not from the compression itself.

6.2 End-to-End Throughput on Phone CPU

Snapdragon-8-Gen-1 (Samsung S22U), Qwen2.5-Coder-3B Q5_K_M target, Qwen2.5-0.5B Q8 draft, `--draft 8`:

Configuration	Eval t/s	Speedup
Vanilla llama.cpp b8861	12.20	1.00×
SP fast path (Phase 1.6/A.2, kcap bypass)	43.72	3.58×

This is on phone CPU only — no GPU, NPU, or HTP offload. Commit `shannon-prime-llama@05c405d`. Spec-decode contributes the majority of the speedup at this size class; the SP fast path provides a further ~30% on top via cache-resident FUSED_KQ.

6.3 A100 Engine Benchmark

RunPod A100, engine-only, 2026-04-25:

Configuration	Tokens/s
Engine baseline	6.34
Engine ship build	7.07 (+11.6%)

The improvement here is below the rate that SP fp8 should provide. Investigation traced this to an engine-side artifact in the fp16 $\rightarrow$ fp32 reduction loop; the SP compression itself contributes more than measured here. This will be addressed in the v1.17 engine release.

6.4 V69 HTP Runtime Graph

Phase 2.5 / 2026-05-02: a MatMul kernel constructed at runtime via `QnnGraph_addNode` with `APP_WRITE` weight injection runs at 238 μ s at 256×256 fp32 on V69 HTP. This is the Mode C dispatch primitive — sufficient to validate that the AOT .bin compile flow is now optional. Per-call FastRPC ceiling: 577 calls/sec.

7. Calibration Status (Model Pack)

Per the in-tree calibration ledger (`docs/MODEL-PACK-CALIBRATION.md`):

Model	Status	PPL Delta vs Vanilla
Phi-3	CALIBRATED	+2.44% (pass)
Qwen3 (edge config)	Edge-fail	+5.14%
Other 6 architectures	PROVISIONAL	not yet calibrated

The Phi-3 calibration is the first published end-to-end result. The Qwen3 edge-fail is a known artifact of the architecture’s gated-attention + mRoPE-mode-8 combination; a fix is scoped for v1.17.

8. Future Work

8.1 Frobenius fp8 Calibration Experiment (Priority)

The Frobenius theorem of §4 predicts that fp8 deployment requires no calibration. The experiment design:

1. Take a Phi-3 model (currently calibrated under the standard ledger).
2. Apply SP encoding at fp16 to all weight and KV tensors.
3. Apply φ_p^8 for $p = 11$ to obtain SP-fp8 representations.
4. Measure PPL on bench corpora *without* any calibration step.
5. Compare to: (a) vanilla Phi-3 fp16, (b) GPTQ-fp8 Phi-3, (c) AWQ-fp8 Phi-3.

Predicted outcome: $|\Delta\text{PPL}| < 1\%$ for SP-fp8 with zero calibration; standard fp8 methods require ~1000 calibration samples to reach the same accuracy.

8.2 Stern–Brocot RoPE

Replace the RoPE base 10,000 with the golden ratio φ in the position-encoding code path; benchmark on long-context tasks (RULER, LongBench). Expected lift on tasks requiring fine positional discrimination. Smallest implementation footprint of any extension.

8.3 Weil Pairing Attention Prototype

Implement the Miller algorithm on CUDA and Hexagon HVX; replace scaled dot-product attention with the Weil pairing on $E[n]$ for n chosen as a small prime. Theoretical complexity: linear in sequence length with $O(\log n)$ per pair; in practice the Miller algorithm has a small constant factor. This is the highest-risk, highest-upside extension.

8.4 Hecke-Eigenform Embeddings

Train a small (1B param) model from scratch with the embedding initialized as Hecke eigenforms of $S_k(\Gamma_0(N))$ for $\dim = d_{\text{embed}}$. Compare to learned embedding at fixed compute. This is the longest-path experiment.

8.5 L-Function Activation Oracle Integration

Wire the L-function multiplicativity into the existing activation oracle. Predicted improvement: tighter prefetch bounds; smaller effective working set at decode time.

9. Related Work

KIVI [Liu et al. 2024] achieves $4\times$ KV compression with measurable perplexity degradation via residual quantization; SP achieves higher compression with provable exactness. StreamingLLM [Xiao et al. 2024] retains a sliding window plus attention sinks; SP retains all positions losslessly. QServe [Lin et al. 2024] is a contemporaneous quantization framework with calibration; SP-fp8 should match QServe accuracy without the calibration step (subject to the §8.1 experiment). DeepSeek-V4’s published architecture [DeepSeek-AI 2026-04-24] independently arrived at KV compression + sliding window + prefetch oracle as a design point, validating the architectural shape SP has been building toward.

10. Conclusion

Shannon-Prime is a transformer compression and acceleration framework grounded in the CM elliptic curve over $\mathbb{Q}(\sqrt{-163})$. Its compression is provably exact, its quantization is structure-preserving by the Frobenius theorem of §4, and its implementation is validated across four backends with end-to-end results including a $3.58\times$ phone-CPU speedup on Qwen2.5-Coder-3B and a $6.2\times$ KV cache compression at zero perplexity delta on Mistral-7B. The framework’s theoretical content is developed in the companion paper [SP-Theory 2026]; this paper has presented the systems contribution.

The single highest-leverage next experiment is the Frobenius-fp8 calibration test of §8.1, which directly probes the framework’s central theoretical claim. We expect it to demonstrate calibration-free fp8 deployment as a deployable capability.

References

(Abbreviated; full bibliography in v0.2.)

- Companion paper: “Shannon-Prime: A CM-Elliptic-Curve Framework for Transformer Computation,” 2026.
- Liu, Z. *et al.* “KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache.” 2024.
- Xiao, G. *et al.* “Efficient Streaming Language Models with Attention Sinks.” 2024.
- Lin, Y. *et al.* “QServe: W4A8KV4 Quantization and System Co-design for Efficient LLM Serving.” 2024.
- DeepSeek-AI. “DeepSeek-V4 Technical Report.” 2026-04-24.
- Su, J. *et al.* “RoFormer: Enhanced Transformer with Rotary Position Embedding.” 2021.
- Silverman, J. H. *The Arithmetic of Elliptic Curves*. Springer GTM 106, 1986.